

Q1.Student Registration System.

(a) Enumerate three Collection interfaces suitable for this system.

- 1.Set – Stores unique students (no duplicates).
- 2.List – Stores multiple enrolled courses for a student.
- 3.Map – Maps each student to their list of courses.

(b) Design a Collection structure (diagram/representation) to store students and their enrolled courses.

HashSet<Student>

|

| (Unique Students)

v

| Alice |

| Bob |

| Charlie |

HashMap<Student, List<String>>

Alice -> [Java, Python]

Bob -> [C, DBMS]

Charlie -> [Java, AI]

(c) Java Program (HashSet + HashMap + Generics)

```
import java.util.*;
```

```
public class StudentRegistrationSystem {  
    public static void main(String[] args) {  
  
        // HashSet to store unique students  
        HashSet<String> students = new HashSet<>();  
  
        students.add("Alice");  
        students.add("Bob");  
        students.add("Charlie");  
  
        // HashMap to map students to courses  
        HashMap<String, ArrayList<String>> studentCourses = new  
        HashMap<>();  
  
        studentCourses.put("Alice", new ArrayList<>(Arrays.asList("Java",  
"Python"))));  
  
        studentCourses.put("Bob", new ArrayList<>(Arrays.asList("C",  
"DBMS"))));  
  
        studentCourses.put("Charlie", new ArrayList<>  
(Arrays.asList("Java", "AI")));  
  
        // Display data using Iterator  
        Iterator<String> iterator = students.iterator();
```

```
System.out.println("Student Registration Details:");
while (iterator.hasNext()) {
    String student = iterator.next();
    System.out.println("Student: " + student);
    System.out.println("Courses: " + studentCourses.get(student));
    System.out.println();
}
}
```

Sample Output

Student Registration Details:

Student: Alice

Courses: [Java, Python]

Student: Bob

Courses: [C, DBMS]

Student: Charlie

Courses: [Java, AI]

Q2: E-Commerce Cart System

(a) Three Collection Classes

1. **ArrayList** – Stores cart items in insertion order.
2. **LinkedList** – Efficient for adding/removing items frequently.
3. **Vector** – Similar to ArrayList but synchronized (thread-safe).

(b) List-Based Structure

Shopping Cart (ArrayList)

Index	Item
-----	-----
0	Laptop
1	Mouse
2	Keyboard
3	Headphones

(c) Java Program (ArrayList + Iterator)

```
import java.util.*;
```

```
public class EcommerceCart {
```

```
    public static void main(String[] args) {
```

```
        // Create ArrayList
```

```
        ArrayList<String> cart = new ArrayList<>();
```

```
// Add items

cart.add("Laptop");

cart.add("Mouse");

cart.add("Keyboard");

cart.add("Headphones");


// Remove an item

cart.remove("Mouse");


// Display all items using Iterator

Iterator<String> iterator = cart.iterator();

System.out.println("Shopping Cart Items:");
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
}
```

Sample Output

Shopping Cart Items:

Laptop

Keyboard

Headphones

Q3: Library Management System

(a) Three Map Implementations in Java

1. HashMap – Fast insertion and retrieval.
2. LinkedHashMap – Maintains insertion order.
3. TreeMap – Stores keys in sorted order.

(b) Map Representation

HashMap<ISBN, Book Name>

9780134685991 → Effective Java

9780071808552 → Java Complete Reference

9780132350884 → Clean Code

(c) Java Program (HashMap)

```
import java.util.*;
```

```
public class LibraryManagement {  
    public static void main(String[] args) {
```

```
        // Create HashMap
```

```
        HashMap<String, String> books = new HashMap<>();
```

```
        // Insert book details
```

```
        books.put("9780134685991", "Effective Java");
```

```
        books.put("9780071808552", "Java Complete Reference");
```

```
books.put("9780132350884", "Clean Code");

// Retrieve a book

String isbn = "9780134685991";

System.out.println("Book with ISBN " + isbn + ": " +
books.get(isbn));


// Display all books

System.out.println("\nLibrary Books:");

for (Map.Entry<String, String> entry : books.entrySet()) {

    System.out.println("ISBN: " + entry.getKey() +

        " | Book: " + entry.getValue());

}

}
```

Sample Output

Book with ISBN 9780134685991: Effective Java

Library Books:

ISBN: 9780134685991 | Book: Effective Java

ISBN: 9780071808552 | Book: Java Complete Reference

ISBN: 9780132350884 | Book: Clean Code

Q4: Employee Data Processing

(a) Three Advantages of Generics in Collections

Type Safety – Prevents storing the wrong type of objects.

No Type Casting – Eliminates the need for explicit casting when retrieving objects.

Code Reusability – Enables the same collection code to work with different object types.

(b) List with Generics Representation

`ArrayList<Employee>`

| Employee(ID:101, Name:John, Salary:45000) |

| Employee(ID:102, Name:Alice, Salary:60000)|

Employee(ID:103, Name:Bob, Salary:75000)

(c) Java Program (Iterator + Generics)

```
import java.util.*;
```

```
class Employee {
```

```
    int id;
```

```
    String name;
```

```
    double salary;
```

```
Employee(int id, String name, double salary) {  
    this.id = id;  
    this.name = name;  
    this.salary = salary;  
}  
}
```

```
public class EmployeeDataProcessing {  
    public static void main(String[] args) {  
  
        // List with Generics  
        ArrayList<Employee> employees = new ArrayList<>();  
  
        // Store employee objects  
        employees.add(new Employee(101, "John", 45000));  
        employees.add(new Employee(102, "Alice", 60000));  
        employees.add(new Employee(103, "Bob", 75000));  
  
        // Display employees with salary > 50000  
        Iterator<Employee> iterator = employees.iterator();  
  
        System.out.println("Employees with Salary > 50000:");
```

```
while (iterator.hasNext()) {  
    Employee emp = iterator.next();  
    if (emp.salary > 50000) {  
        System.out.println("ID: " + emp.id);  
        System.out.println("Name: " + emp.name);  
        System.out.println("Salary: " + emp.salary);  
        System.out.println();  
    }  
}  
}
```

Sample Output

Employees with Salary > 50000:

ID: 102

Name: Alice

Salary: 60000.0

ID: 103

Name: Bob

Salary: 75000.0

Q5: Online Voting System

(a) Three Suitable Collection Types

1. HashSet – Stores unique voter IDs and prevents duplicate voting.
2. HashMap – Stores candidates and their vote counts.
3. TreeMap – Displays vote results in sorted order (optional).

(b) Structure Using Set and Map

HashSet<VoterID>

V101

V102

V103

|

| (Unique voters)

v

HashMap<Candidate, Vote Count>

Alice → 2

Bob → 1

Charlie → 3

(c) Java Program (HashSet + HashMap)

```
import java.util.*;
```

```
public class OnlineVotingSystem {
```

```
    public static void main(String[] args) {
```

```
        // Store unique voters
```

```
        HashSet<String> voters = new HashSet<>();
```

```
        // Store vote count
```

```
        HashMap<String, Integer> votes = new HashMap<>();
```

```
        // Record votes
```

```
        castVote("V101", "Alice", voters, votes);
```

```
        castVote("V102", "Bob", voters, votes);
```

```
        castVote("V103", "Alice", voters, votes);
```

```
        castVote("V101", "Charlie", voters, votes); // Duplicate vote
```

```
        // Display results
```

```
        System.out.println("\nVoting Results:");
```

```
        for (Map.Entry<String, Integer> entry : votes.entrySet()) {
```

```
            System.out.println(entry.getKey() + " : " + entry.getValue());
```

```
        }
```

```
    }
```

```
public static void castVote(String voterId, String candidate,
                             HashSet<String> voters,
                             HashMap<String, Integer> votes) {

    if (voters.contains(voterId)) {
        System.out.println("Duplicate vote not allowed for " + voterId);
    } else {
        voters.add(voterId);
        votes.put(candidate, votes.getOrDefault(candidate, 0) + 1);
        System.out.println(voterId + " voted for " + candidate);
    }
}
}
```

Sample Output

V101 voted for Alice

V102 voted for Bob

V103 voted for Alice

Duplicate vote not allowed for V101

Voting Results:

Alice : 2

Bob : 1

